

Introducción a JavaScript

¿Qué es JavaScript?

JavaScript es un lenguaje de programación interpretado, dinámico y de alto nivel que fue diseñado inicialmente para hacer las páginas web más interactivas. Aunque comenzó como un lenguaje para el navegador, hoy en día se utiliza en muchos otros entornos.

Características principales de JavaScript

- **Interpretado:** No necesita compilación, se ejecuta directamente
- **Dinámico:** Los tipos de datos se determinan en tiempo de ejecución
- **Orientado a objetos:** Soporta programación orientada a objetos
- **Funcional:** Las funciones son "ciudadanos de primera clase"
- **Débilmente tipado:** No es necesario declarar el tipo de las variables

JavaScript vs Java

A pesar del nombre similar, **JavaScript y Java son lenguajes completamente diferentes:**

JavaScript	Java
Interpretado	Compilado
Tipado dinámico	Tipado estático
Ejecución en navegador	Ejecución en máquina virtual
Sintaxis más flexible	Sintaxis más rígida

Historia de JavaScript

- **1995:** Brendan Eich crea JavaScript en Netscape en 10 días
- **1997:** Estandarización como ECMAScript (ES1)
- **1999:** ES3 - Primera versión ampliamente adoptada
- **2009:** ES5 - Muchas mejoras importantes
- **2015:** ES6/ES2015 - Revolución del lenguaje

- **2016-presente:** Actualizaciones anuales (ES2016, ES2017, etc.)

Cómo desarrollar y probar código JavaScript

Para desarrollar y probar código JavaScript, existen varias herramientas y entornos disponibles:

- **Navegadores web:** Todos los navegadores modernos tienen consolas de desarrollo integradas (Chrome DevTools, Firefox Developer Tools, etc.) donde se puede escribir y probar código JavaScript directamente.
- **Editores de código:** Herramientas como Visual Studio Code, Sublime Text o Atom ofrecen soporte avanzado para JavaScript con resaltado de sintaxis, autocompletado y depuración.
- **Entornos de ejecución:** Node.js permite ejecutar JavaScript fuera del navegador, ideal para desarrollo del lado servidor y scripts de automatización.

Para este curso, se recomienda utilizar **Visual Studio Code** junto con la consola del navegador para probar y depurar el código JavaScript.

Cómo ejecutar código JavaScript en el navegador

La consola del navegador es una herramienta fundamental para depurar JavaScript. Para usarla:

- Abre las herramientas de desarrollo (F12 o clic derecho > "Inspeccionar").
- Ve a la pestaña "Consola".
- Escribe código JavaScript directamente y presiona Enter para ejecutarlo.

```
// Diferentes tipos de mensajes en consola
console.log("Mensaje informativo");
console.warn("Advertencia");
console.error("Error");
console.info("Información adicional");

// Mostrar datos en formato tabla
console.table([
  {nombre: "Ana", edad: 25},
  {nombre: "Luis", edad: 30}
]);

// Agrupar mensajes
console.group("Datos del usuario");
console.log("Nombre: Ana");
console.log("Edad: 25");
console.groupEnd();
```



```
// Medir tiempo de ejecución
console.time("operacion");
// ... código a medir ...
console.timeEnd("operacion");
```

Cómo desarrollar Javascript con Visual Studio Code

Visual Studio Code (VSCode) es actualmente el editor de código más popular para desarrollo JavaScript debido a su excelente soporte nativo y su rico ecosistema de extensiones.

Características nativas de VSCode para JavaScript

IntelliSense integrado: VSCode proporciona autocompletado inteligente, información de parámetros y documentación automática para JavaScript sin necesidad de configuración adicional.

```
// VSCode mostrará automáticamente las propiedades y métodos disponibles
let texto = "Hola mundo";
texto. // Aquí aparecerá el autocompletado con métodos como .length, .toUpperCase
```

Depuración integrada: Permite establecer breakpoints, inspeccionar variables y ejecutar código paso a paso directamente en el editor.

Terminal integrado: Ejecuta comandos de Node.js y herramientas de desarrollo sin salir del editor.

Extensiones esenciales para JavaScript

1. **ESLint** - Análisis de código y detección de errores en tiempo real
2. **Prettier - Code formatter** - Formateo automático de código
3. **JavaScript (ES6) code snippets** - Fragmentos de código predefinidos
4. **Live Server** - Servidor web local para desarrollo

Configuración recomendada

Para optimizar VSCode para JavaScript, crea un archivo `settings.json` en tu proyecto:

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  }
}
```

```
},
"javascript.suggest.autoImports": true,
"typescript.suggest.autoImports": true,
"editor.tabSize": 2,
"editor.insertSpaces": true
}
```

Configuración de Prettier (archivo `.prettierrc`):

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 80
}
```



Atajos de teclado útiles

- `Ctrl + Shift + P`: Paleta de comandos (acceso a todas las funciones)
- `Ctrl + ñ`: Abrir/cerrar terminal integrado
- `F5`: Iniciar depuración
- `F9`: Establecer/quitar breakpoint
- `F12`: Ir a definición
- `Shift + F12`: Mostrar todas las referencias
- `Ctrl + D`: Seleccionar siguiente ocurrencia
- `Alt + Shift + F`: Formatear documento
- `Ctrl + /`: Comentar/descomentar línea

Depuración en VSCode

Para depurar JavaScript en el navegador, crea un archivo `.vscode/launch.json` en tu proyecto:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "chrome",
      "request": "launch",
```



```

        "name": "Launch Chrome against localhost",
        "url": "http://localhost:5500",
        "webRoot": "${workspaceFolder}",
        "sourceMaps": true
    },
    {
        "type": "node",
        "request": "launch",
        "name": "Launch Node.js",
        "program": "${workspaceFolder}/app.js",
        "console": "integratedTerminal"
    }
]
}

```

Snippets útiles

VSCode permite crear **snippets personalizados**. Ve a `File > Preferences > Configure User Snippets` y selecciona JavaScript:

```

{
  "Console Log": {
    "prefix": "cl",
    "body": [
      "console.log($1);"
    ],
    "description": "Console log"
  },
  "Arrow Function": {
    "prefix": "af",
    "body": [
      "const $1 = ($2) => {",
      "\t$3",
      "};"
    ],
    "description": "Arrow function"
  }
}

```

Ejemplo de uso completo en VSCode

1. **Crear proyecto:** crea una carpeta para tu proyecto y ábrela en VSCode. En la consola:

```
mkdir mi-proyecto-js
cd mi-proyecto-js
code . # Abre VSCode
```

1. **Estructura del proyecto:** la estructura recomendada es:

```
mi-proyecto-js/
├── .vscode/
│   ├── settings.json
│   └── launch.json
├── src/
│   ├── index.html
│   ├── script.js
│   └── styles.css
├── .eslintrc.json
├── .prettierrc
└── package.json
```

1. **Archivo HTML básico:**

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Proyecto JavaScript</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Mi Aplicación</h1>
  <script src="script.js"></script>
</body>
</html>
```

Con esta configuración, tendrás un entorno de desarrollo para JavaScript con:

- Formateo automático
- Detección de errores
- Autocompletado inteligente
- Depuración integrada
- Control de versiones con Git

Sintaxis básica de JavaScript

A continuación, se presentan los conceptos básicos de la sintaxis de JavaScript con ejemplos prácticos.

Incluir JavaScript en HTML

Para añadir funcionalidad JavaScript a una página web, se puede incluir el código de dos formas principales:

Mediante etiqueta script interna

Añadimos un bloque de código JavaScript directamente dentro de la etiqueta `<script>` en el HTML.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi página</title>
</head>
<body>
  <h1>Hola Mundo</h1>

  <script>
    console.log("¡Hola desde JavaScript!");
  </script>
</body>
</html>
```



Mediante archivo externo

Referenciamos un archivo `.js` externo usando la etiqueta `<script>` con el atributo `src`.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi página</title>
</head>
<body>
  <h1>Hola Mundo</h1>

  <script src="mi-script.js"></script>
</body>
```



```
</html>
```

¿Cual es la mejor práctica?

Es recomendable colocar las etiquetas `<script>` justo antes del cierre de la etiqueta `</body>`. Esto asegura que el HTML se cargue primero, mejorando el rendimiento y evitando errores si el script intenta acceder a elementos del DOM que aún no existen.

Variables en JavaScript

Las variables se utilizan para almacenar datos que pueden cambiar durante la ejecución del programa. En JavaScript, existen tres formas principales de declarar variables: `var`, `let` y `const`.

var, let y const

```
// var - forma tradicional (evitar en código nuevo)
var nombre = "Ana";

// let - para variables que pueden cambiar
let edad = 25;
edad = 26; // Podemos cambiar el valor

// const - para constantes (no pueden cambiar)
const PI = 3.14159;
// PI = 3.14; // ❌ Error: no se puede cambiar
```



Reglas de nomenclatura

Debemos seguir ciertas reglas al nombrar variables:

```
// ✅ Correcto
let nombre;
let nombreCompleto;
let edad2;
let _precio;
let $total;

// ❌ Incorrecto
let 2nombre;           // No empezar por número
let nombre-completo;   // No guiones
let class;             // Palabra reservada
```



Tipos de datos

Tipos primitivos

Son los tipos de datos básicos en JavaScript:

- `String` (cadenas de texto)
- `Number` (números, tanto enteros como decimales)
- `Boolean` (verdadero/falso)
- `undefined` (sin definir)
- `null` (valor nulo intencionado). El valor nulo representa la ausencia intencionada de cualquier valor u objeto.

```
// String (cadenas de texto)
let nombre = "María";
let apellido = 'González';
let mensaje = `Hola, ${nombre}`; // Template Literals

// Number (números)
let entero = 42;
let decimal = 3.14;
let negativo = -5;

// Boolean (verdadero/falso)
let esVerdad = true;
let esFalso = false;

// undefined (sin definir)
let sinValor;
console.log(sinValor); // undefined

// null (valor nulo intencionado)
let valorNulo = null;
```



Verificar tipos

En JavaScript, podemos verificar el tipo de una variable usando `typeof`, y conocer si una variable es de un tipo específico.

```
let edad = 25;
let nombre = "Ana";
```



```
console.log(typeof edad); // "number"
console.log(typeof nombre); // "string"
```

Operadores

Los operadores son las acciones que podemos realizar sobre los datos. A continuación, se presentan los operadores más comunes en JavaScript.

Operadores aritméticos

Son operadores que realizan operaciones matemáticas básicas. Utilizan números como operandos y devuelven números como resultado.

```
let a = 10;
let b = 3;

console.log(a + b); // 13 - Suma
console.log(a - b); // 7 - Resta
console.log(a * b); // 30 - Multiplicación
console.log(a / b); // 3.333... - División
console.log(a % b); // 1 - Módulo (resto)
console.log(a ** b); // 1000 - Exponente (10³)

// Operadores de incremento/decremento
let contador = 0;
contador++; // Incrementa en 1
contador--; // Decrementa en 1
```



Operadores de comparación

Permiten comparar valores y devuelven un valor booleano (`true` o `false`). Es importante comparar objetos o variables del mismo tipo para evitar resultados inesperados.

```
let x = 5;
let y = "5";

// Igualdad (compara valor, convierte tipos)
console.log(x == y); // true

// Igualdad estricta (compara valor y tipo)
console.log(x === y); // false
```



```
// Desigualdad
console.log(x != y); // false
console.log(x !== y); // true

// Comparaciones numéricas
console.log(x > 3); // true
console.log(x < 10); // true
console.log(x >= 5); // true
console.log(x <= 4); // false
```

Operadores lógicos

Los operadores lógicos se utilizan para combinar expresiones booleanas.

```
let tieneLicencia = true;
let tieneCoche = false;
let edad = 20;

// AND (&&) - ambos deben ser true
let puedeConducir = tieneLicencia && edad >= 18; // true

// OR (||) - al menos uno debe ser true
let puedeViajar = tieneCoche || tieneLicencia; // true

// NOT (!) - invierte el valor
let noTieneCoche = !tieneCoche; // true
```



Estructuras de control

Las estructuras de control permiten dirigir el flujo de ejecución del código según ciertas condiciones o repeticiones.

Condicionales

```
let edad = 18;

// if simple
if (edad >= 18) {
  console.log("Eres mayor de edad");
}

// if-else
if (edad >= 18) {
```



```

    console.log("Puedes votar");
} else {
    console.log("No puedes votar aún");
}

// if-else if-else
let nota = 8;
if (nota >= 9) {
    console.log("Sobresaliente");
} else if (nota >= 7) {
    console.log("Notable");
} else if (nota >= 5) {
    console.log("Aprobado");
} else {
    console.log("Suspenso");
}

// switch
let dia = "lunes";
switch (dia) {
    case "lunes":
        console.log("Inicio de semana");
        break;
    case "viernes":
        console.log("Fin de semana");
        break;
    default:
        console.log("Día normal");
}

```

Operador ternario

Es una alternativa compacta al `if-else` para asignar valores basados en una condición.

```

let edad = 20;
// Operador ternario: lo podríamos leer: "Si edad es mayor o igual a 18, mensaje
let mensaje = edad >= 18 ? "Mayor de edad" : "Menor de edad";
console.log(mensaje); // "Mayor de edad"

```

Bucles

```

// for - cuando sabemos las repeticiones
for (let i = 0; i < 5; i++) {

```

```

    console.log("Número: " + i);
}

// while - mientras se cumpla una condición
let contador = 0;
while (contador < 3) {
    console.log("Contador: " + contador);
    contador++;
}

// do-while - ejecuta al menos una vez
let numero = 0;
do {
    console.log("Número: " + numero);
    numero++;
} while (numero < 3);

```

Funciones

Las funciones son bloques de código reutilizables que realizan una tarea específica. Pueden recibir entradas (parámetros) y devolver un resultado. Nos referiremos a ellas mediante el nombre asignado. Al emplear las funciones tenemos que tener en cuenta:

- **Declaración** de la función: es el proceso de definir una función con un nombre y un bloque de código. En este paso indicamos el nombre de la función, los parámetros (opcionales) que recibe, el valor de retorno (si lo tiene) y lo que debe hacer la función (cuerpo de la función).
- **Llamada o invocación** de la función: es el proceso de ejecutar la función previamente declarada. Al llamar a la función, se utiliza su nombre seguido de paréntesis, y si la función acepta parámetros, se pasan los valores correspondientes dentro de esos paréntesis. Si la función devuelve un retorno, podemos asignarla a una variable o usarla directamente.

En Javascript, utilizamos la palabra reservada `function` para declarar una función. Para devolver un valor, usamos la palabra reservada `return`.

Declaración de funciones

```

// Función básica. La función se llama "saludar", y no recibe parámetros
function saludar() {
    console.log("¡Hola!");
}

// Llamar la función
saludar();

```

```
// Función con parámetros. La función recibe un parámetro "nombre". Este parámetro
function saludarPersona(nombre) {
    console.log("¡Hola, " + nombre + "!");
}

saludarPersona("Ana");

// Función que retorna un valor. Esta función recibe dos parámetros y devuelve su suma
function sumar(a, b) {
    return a + b;
}

let resultado = sumar(5, 3);
console.log(resultado); // 8
```

Diferentes formas de crear funciones

Como se mencionó anteriormente, una forma de crear funciones es definiéndolas mediante la palabra reservada `function`. Sin embargo, existen otras formas de definir funciones en JavaScript:

- **Expresión de función:** Asignar una función anónima a una variable. Decimos que la función es "anónima" porque no tiene un nombre propio, sino que se accede a ella a través de la variable a la que está asignada.
- **Arrow function:** Introducida en ES6, es una forma más concisa de escribir funciones. No tiene su propio `this`, lo que puede ser útil en ciertos contextos. Si la función tiene una sola expresión, se puede omitir el bloque `{}` y el `return`.

```
// 1. Declaración de función
function miFuncion1() {
    return "Soy una declaración";
}

// 2. Expresión de función
let miFuncion2 = function() {
    return "Soy una expresión";
};

// 3. Arrow function (función flecha) - ES6. Esta función no tiene parámetros (puede tenerlos)
let miFuncion3 = () => {
    return "Soy una arrow function";
};

// Arrow function simplificada La función tiene dos parámetros y una sola expresión
let miFuncion4 = (a, b) => a + b;
```

```
let sumar = (a, b) => a + b;

// Llamadas
console.log(miFuncion1()); // "Soy una declaración"
console.log(miFuncion2()); // "Soy una expresión"
console.log(miFuncion3()); // "Soy una arrow function"
console.log(sumar(2, 3)); // 5
```

Parámetros por defecto

Podemos asignar valores por defecto a los parámetros de una función. Si no se proporciona un valor al llamar a la función, se utilizará el valor por defecto.

```
function saludar(nombre = "Usuario") {
    console.log("¡Hola, " + nombre + "!");
}

saludar();           // "¡Hola, Usuario!"
saludar("Ana");      // "¡Hola, Ana!"
```



Arrays (Arreglos)

Un array es una **colección ordenada** de elementos que pueden ser de cualquier tipo (números, cadenas, objetos, etc.). Los arrays en JavaScript son dinámicos y pueden cambiar de tamaño.

Accedemos a los elementos mediante su **índice** (posición), que comienza en 0.

Crear y usar arrays

```
// Crear arrays
let frutas = ["manzana", "banana", "naranja"];
let numeros = [1, 2, 3, 4, 5];
let mixto = ["texto", 42, true, null];

// Acceder a elementos (índice empieza en 0)
console.log(frutas[0]); // "manzana"
console.log(frutas[1]); // "banana"

// Modificar elementos
frutas[1] = "kiwi";
console.log(frutas); // ["manzana", "kiwi", "naranja"]

// Propiedades y métodos útiles
```



```
console.log(frutas.length);    // 3
frutas.push("uva");           // Añadir al final
frutas.pop();                 // Quitar del final
frutas.unshift("pera");       // Añadir al inicio
frutas.shift();               // Quitar del inicio
```

Recorrer arrays

Tenemos diferentes alternativas para recorrer los elementos de un array.

```
let numeros = [1, 2, 3, 4, 5];

// Con for tradicional
for (let i = 0; i < numeros.length; i++) {
  console.log(numeros[i]);
}

// Con for...of (más moderno)
for (let numero of numeros) {
  console.log(numero);
}

// Con forEach
numeros.forEach(function(numero) {
  console.log(numero);
});
```



Objetos

Un objeto es una colección de propiedades, donde cada propiedad es una asociación entre un nombre (clave) y un valor. Los objetos permiten agrupar datos relacionados y funcionalidades (métodos) en una sola entidad. Podemos asociarlo a los objetos de Java o a los diccionarios de Python.

Crear y usar objetos

```
// Crear un objeto
let persona = {
  nombre: "Ana",
  edad: 25,
  esEstudiante: true,
  hobbies: ["leer", "nadar", "viajar"]
}
```



```
};

// Acceder a propiedades
console.log(persona.nombre);    // "Ana"
console.log(persona["edad"]);   // 25

// Modificar propiedades
persona.edad = 26;
persona["esEstudiante"] = false;

// Añadir nuevas propiedades
persona.ciudad = "Madrid";

// Eliminar propiedades
delete persona.hobbies;
```

Métodos en objetos

Los métodos son funciones que están asociadas a un objeto. Se definen como propiedades cuyo valor es una función.

```
let calculadora = {
  sumar: function(a, b) {
    return a + b;
  },
  restar: function(a, b) {
    return a - b;
  }
};

console.log(calculadora.sumar(5, 3)); // 8
console.log(calculadora.restar(5, 3)); // 2
```



Sintaxis moderna de objetos (ES6)

Desde ES6, podemos usar una sintaxis más concisa para definir propiedades y métodos en objetos cuando el nombre de la propiedad es el mismo que el nombre de la variable o cuando definimos métodos.

```
let nombre = "Ana";
let edad = 25;

// Sintaxis abreviada
```



```
let persona = {
  nombre, // Equivale a nombre: nombre
  edad,   // Equivale a edad: edad

  // Método abreviado
  saludar() {
    console.log(`Hola, soy ${this.nombre}`);
  }
};
```

Conceptos avanzados básicos

Template literals

Los *template literals* son una forma de trabajar con cadenas de texto en JavaScript. Se definen utilizando comillas invertidas (```) y permiten incluir expresiones dentro de la cadena.

```
let nombre = "Ana";
let edad = 25;

// Forma tradicional
let mensaje1 = "Hola, soy " + nombre + " y tengo " + edad + " años";

// Con template literals (más legible)
let mensaje2 = `Hola, soy ${nombre} y tengo ${edad} años`;

// Multilínea
let html = `
  <div>
    <h1>${nombre}</h1>
    <p>Edad: ${edad}</p>
  </div>
`;
```



Destructuring

El *destructuring* es una expresión que permite desempaquetar valores de arrays o propiedades de objetos en variables distintas. Es una forma de "deconstruir" estructuras complejas de datos de manera compacta y legible.

```
// Array destructuring
```



```

let numeros = [1, 2, 3];
let [primero, segundo, tercero] = numeros;

console.log(primero); // 1
console.log(segundo); // 2

// Object destructuring
let persona = { nombre: "Ana", edad: 25 };
let { nombre, edad } = persona;

console.log(nombre); // "Ana"
console.log(edad); // 25

```

Spread operator

El *spread operator* (`...`) permite expandir elementos de un array u objeto en lugares donde se esperan múltiples elementos o propiedades. Es útil para copiar, combinar o clonar arrays y objetos.

- Con arrays, permite combinar varios arrays en uno solo.
- Con objetos, permite copiar propiedades de un objeto a otro o combinar varios objetos.

```

// Con arrays
let numeros1 = [1, 2, 3];
let numeros2 = [4, 5, 6];
let todos = [...numeros1, ...numeros2]; // [1, 2, 3, 4, 5, 6]

// Con objetos
let persona = { nombre: "Ana", edad: 25 };
let empleado = { ...persona, trabajo: "Programadora" };
// { nombre: "Ana", edad: 25, trabajo: "Programadora" }

// Combinar dos objetos
let obj1 = { a: 1, b: 2 };
let obj2 = { b: 3, c: 4 };
let combinado = { ...obj1, ...obj2 }; // { a: 1, b: 3, c: 4 }. El valor de 'b' s

```

Buenas prácticas en JavaScript

1. Usa nombres descriptivos

```
// ❌ MaL
```

```
let x = getUserAge();
let d = new Date();

// ✅ Bien
let userAge = getUserAge();
let currentDate = new Date();
```

2. Usa const por defecto, let cuando necesites cambiar

```
// ✅ Bien
const API_URL = "https://api.ejemplo.com";
const users = getUsers();

let currentIndex = 0; // Solo let si va a cambiar
```



3. Usa funciones puras cuando sea posible

```
// ❌ Función impura (modifica variables externas)
let total = 0;
function sumarAlTotal(cantidad) {
  total += cantidad;
}

// ✅ Función pura (no efectos secundarios)
function sumar(a, b) {
  return a + b;
}
```



4. Maneja errores apropiadamente

```
function dividir(a, b) {
  if (b === 0) {
    throw new Error("No se puede dividir por cero");
  }
  return a / b;
}

try {
  let resultado = dividir(10, 0);
  console.log(resultado);
} catch (error) {
```



```
    console.log("Error:", error.message);  
  }  
}
```

Ejercicios prácticos

Ejercicio 1: Calculadora básica

```
function calculadora(num1, num2, operacion) {  
  switch (operacion) {  
    case '+':  
      return num1 + num2;  
    case '-':  
      return num1 - num2;  
    case '*':  
      return num1 * num2;  
    case '/':  
      return num2 !== 0 ? num1 / num2 : "Error: División por cero";  
    default:  
      return "Operación no válida";  
  }  
}  
  
console.log(calculadora(10, 5, '+')); // 15  
console.log(calculadora(10, 5, '*')); // 50
```



Ejercicio 2: Validador de edad

```
function validarEdad(edad) {  
  if (typeof edad !== 'number' || edad < 0) {  
    return "Edad no válida";  
  }  
  
  if (edad < 18) {  
    return "Menor de edad";  
  } else if (edad >= 18 && edad < 65) {  
    return "Adulto";  
  } else {  
    return "Adulto mayor";  
  }  
}  
  
console.log(validarEdad(25)); // "Adulto"
```



```
console.log(validarEdad(10)); // "Menor de edad"
console.log(validarEdad(70)); // "Adulto mayor"
```

Ejercicio 3: Gestión de estudiantes

```
let estudiantes = [
  { nombre: "Ana", edad: 20, notas: [8, 9, 7] },
  { nombre: "Luis", edad: 19, notas: [6, 7, 8] },
  { nombre: "Carmen", edad: 21, notas: [9, 9, 10] }
];

function calcularPromedio(notas) {
  let suma = notas.reduce((total, nota) => total + nota, 0);
  return suma / notas.length;
}

function mostrarEstudiantes() {
  estudiantes.forEach(estudiante => {
    let promedio = calcularPromedio(estudiante.notas);
    console.log(`${estudiante.nombre}: Promedio ${promedio.toFixed(2)}`);
  });
}

mostrarEstudiantes();
// Ana: Promedio 8.00
// Luis: Promedio 7.00
// Carmen: Promedio 9.33
```

Herramientas de desarrollo

A la hora de desarrollar en JavaScript, existen varias herramientas que facilitan la depuración y el análisis del código.

Próximos pasos

Ahora que conoces los fundamentos de JavaScript, estás preparado para:

1. **Interactuar con el DOM** - Manipular elementos HTML
2. **Manejar eventos** - Responder a clics, teclas, etc.
3. **Trabajar con APIs** - Obtener datos externos
4. **Usar frameworks** - React, Vue, Angular

5. **Desarrollar aplicaciones completas**

JavaScript es un lenguaje muy potente y versátil. Con estos fundamentos, puedes empezar a crear aplicaciones web interactivas y continuar aprendiendo conceptos más avanzados según tus necesidades.